Document number: P2042r0
Date: 2020-01-11
Audience: LEWG
Reply-to: *Andrzej Krzemieński <akrzemi1 at gmail dot com>*

Alternate names for `make_shared_default_init`

This document is a response to National body comment [DE002], which requests to rename functions `make_shared_default_init`, `make_unique_default_init` and `allocate_shared_default_init`, as there exist users that incorrectly infer these functions' semantics from their names. In this paper we list the suggested alternate names for these functions, which have been collected from the reflector discussions.

Naming criteria

The following (potentially contradictory) criteria have been expressed for the desired name of these functions:

1. Name should be consistent with the original names, as used in Boost implmentation (`make_shared_noinit`).
2. Name should be consistent with the core definitions. (Term "default initialization" is already defined in the Standard with exactly the semantics in `make_shared_default_init`.)
3. Name should be consistent with other names in the Standard Library. (We already have `std::uninitialized_default_construct`.)
4. Name should not use "init" or "initialize" as to some people this implies that the objects will be left in a state that can be inspected without cassing undefined behavior.
5. Name should reflect the intent of the function, which is, "perform as fast initialization as possible, knowing that the next thing that will happen with this values will be to overwrite them."
6. If an ideal name cannot be found, prefer a name that is confusing (the user realizes that she doesn't know what the function is doing) to the name that is misleading (the user is led to believe that she knows what the function is doing, whereas the function is actually doing something else).

The names

`make_shared_default_init`

This name is consistent with core language and more-less (`make_shared_default_init` vs `make_shared_default_construct`) with the Standard Library. The only problem is the issue inidcated in [DE002], that for people not familiar with core terms word "init" is misleading as it implies a state that can be read.

`make_shared_default_construct`

Almost same as above, but we get a 100% compatibility with the Standard Library.

`make_shared_uninitialized` (+ a constraint)

This is what [P1973r0] proposes. This avoids using the name "initialized" and does not mislead people who infer semantics from function name. To some extent the name reflects what the function does, because for the types that the function is constrained to, there is little difference between being initialized an uninitialized. But this solution has its own issues. First, in addition to name, it changes the design, so that the usage of the function is impeded in generic contexts. The name no longer reflects the core terms or the Standard Library names. It is also misleading, this time to another group of people, who follow the core terms. "Uninitialized", as in `std::uninitialized_default_construct`, implies that we are left with raw memory and some initialization — even if vacuous initialization — still needs to be performed by the caller.

`make_shared_minimal_init`

This is what [P1978r0] proposes. Arguably, its name reflects the intention somewhat better, as the purpose of invoking this function is to perform as fast initialization as possible for the subsequent value assignment. But it departs from core terms, and does not actually address [DE002], because the misleading term "init" is still present.

`make_shared_noinit`

This is the name used in the original implementation in Boost. It addresses [DE002] as "noinit" implies that no "initialization" is performed. Drawbacks: it doesn't use core terms, and is somewhat misleading as some it might indicate that not even vacuous initialization has been performed.

`make_shared_unsafe_init`, `make_shared_weak_init`

These names have prefixes that alert about something fishy, but they still have "init", which is potentially misleading. They do not reflect core terms. They also don't really reflect what the function does. Unless we introduce a new core term "weak initialization".

`make_shared_trivial_init`

Same as above, except a bit more misleading, as it implies that it only works for trivial types, or that it skips the constructor for types that have one.

`make_shared_nonvalue_init`

"Nonvalue" hints that the produced value may be fishy, but it is not clear if it hints enough that it cannot be read. Still, not following core terms, and has "init".

`make_shared_partially_formed_value`, `make_shared_with_partially_formed_value`

"Partially formed value" sounds dangerous enough to discourage people form reading the value. It doesn't contain "init". It reflects the intent a bit. But it is not actually true because for types with default constructor the value is actually fully formed.

`make_shared_for_overwrite`, `make_shared_with_no_guaranteed_value`

Clearly reflect the intent. Do not have "init". Do not invent terms. Maybe too descriptive.

`make_shared_unspecified_value`

Uses familiar core terms, maybe somewhat libelarly. Still, the uninitiated may think that unspecified value can be read.

`make_shared_valueless`, `make_shared_with_valueless`

"Valueless" is used in `std::variant` and is perhaps sufficiently scarry. The two contexts are similar in the sense that in either case you do not want to read the value, but to overwrite it.

`make_shared_with_discarded_value`, `make_shared_discarded_value`

You do not want to read a discarded value, don't you?

`make_shared_indeterminate_value`, `make_shared_with_indeterminate_value`, make_shared_indeterminate

This uses the core term "indeterminate value" which is the right term at least for trivial types. It is not the right term for types with default constructor, but the meaning should be sufficiently clear from the context. This makes it incompatible with `std::uninitialized_default_construct`.

Other options

The reflector discussions also suggested that the core terms themselves could be changed to be easier to consume by the uninitiated. However, this would require the coordination between the working groups that may not be feasible for C++20. So, either this idea needs to be abandoned, or LEWG can preemptively invent intuitive terms in hopes that CWG will adapt in the future.

The feature test macro

It should be noted that `std::make_shared_default_init` is acompanied by the corresponding feature test macro `__cpp_lib_smart_ptr_default_init`. Should LEWG decide to change the name of the function, does the name of the macro also require a change?

References

1. [DE002], "Rename make_unique/shared_default_init to make_unique/shared_nonvalue_init", (DE002, https://github.com/cplusplus/nbballot/issues/2).
2. [P1973r0], Nicolai Josuttis, "Rename '_default_init' Functions", (http://wiki.edg.com/pub/Wg21belfast/LibraryEvolutionWorkingGroup/D1973R0_rename_default_init_funcs.pdf).
3. [P1020r1], Glen Joseph Fernandes, Peter Dimov, "Smart pointer creation with default initialization", (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1020r1.html).
4. Greg Colvin, Beman Dawes, Peter Dimov, Glen Fernandes, Boost.SmartPtr (https://www.boost.org/doc/libs/1_71_0/libs/smart_ptr/doc/html/smart_ptr.html#introduction).
5. [P1978r0], Andrzej Krzemieński, Glen Joseph Fernandes, Nevin Liber, Peter Dimov, "Rename `make_shared_default_init` functions and do nothing more", (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1978r0.html).